LAB 3 · WEEKS 5–6

Data Protection: Encryption & Key Management

At-rest & in-transit encryption, envelope encryption and cryptographic erasure — OpenSSL & LocalStack KMS

Lab Learning Outcomes

At the end of this lab, you will be able to:

1. Encrypt and decrypt data with **symmetric** (AES) and **asymmetric** (RSA) cryptography.
2. Protect data **in transit** with TLS and observe the difference between plaintext and encrypted traffic.
3. Use a **Key Management Service (KMS)** and implement **envelope encryption**.
4. Apply **per-tenant keys** and perform **cryptographic erasure** to make data provably unrecoverable.
5. Verify **data integrity** with hashing and build a tamper-evident (hash-chained) record.

Course & Assessment Mapping

Item	Mapping
Course Learning Outcome	CLO2 — Construct secure cloud operations that safeguard data integrity (VBE3)
Lecture topics	Week 4 (Data Protection) · reinforced by Week 9 (Key Management patterns)
Value / skill clusters	VBE3 (Integrity) · SC8 (Integrated Problem-Solving)
Assessment	Lab report (outputs + short answers) — contributes to the Lab Assignment

Lab Arrangement (2 Sessions over 2 Weeks)

Session	Week	Focus
Session A	Week 5	Symmetric/asymmetric encryption; data at rest and in transit (Tasks 1–3)
Session B	Week 6	KMS, envelope encryption, per-tenant keys, cryptographic erasure, integrity (Tasks 4–7), then the report

Note: Session A builds the cryptographic fundamentals by hand. Session B shows how a cloud KMS manages keys at scale and enables provable deletion.

Technical Prerequisites

- A laptop with Docker and a terminal.
- **OpenSSL** — pre-installed on macOS/Linux; on Windows use Git Bash or WSL.
- **AWS CLI v2** pointed at **LocalStack** (as in Lab 1) for the KMS tasks.
- Basic comfort with the command line.

> **Security tip:** Encryption is only as strong as its key management. Watch carefully in Session B where the keys live — that is the real security control, not the algorithm.

Session A (Week 5) — Encryption Fundamentals

Task 1 — Symmetric Encryption (Data at Rest)

Create a sensitive file and encrypt it with AES-256. Then decrypt it. One shared key does both — fast, but the key must be protected.

```
# Create a sample sensitive record
echo 'Patient: Ahmad, Diagnosis: confidential' > record.txt

# Encrypt with AES-256 (you will be prompted for a passphrase = the key)
openssl enc -aes-256-cbc -pbkdf2 -salt -in record.txt -out record.enc

# Prove it is unreadable
cat record.enc

# Decrypt back
openssl enc -d -aes-256-cbc -pbkdf2 -in record.enc -out record.dec.txt
diff record.txt record.dec.txt && echo 'MATCH: decryption successful'
```

In your report: what is the key-distribution problem with symmetric encryption, and why does it matter for the cloud?

Task 2 — Asymmetric Encryption & Digital Signatures

Generate an RSA key pair. Anyone can encrypt with the public key; only the private key decrypts. Signatures prove origin and integrity.

```
# Generate a 2048-bit key pair
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem

# Encrypt with the PUBLIC key, decrypt with the PRIVATE key
openssl pkeyutl -encrypt -pubin -inkey public.pem -in record.txt -out record.rsa
openssl pkeyutl -decrypt -inkey private.pem -in record.rsa -out record.rsa.txt

# Sign with the PRIVATE key; verify with the PUBLIC key
openssl dgst -sha256 -sign private.pem -out record.sig record.txt
openssl dgst -sha256 -verify public.pem -signature record.sig record.txt
```

> **Note:** Note how the roles reverse: encryption uses the public key, signing uses the private key. This is the basis of PKI and TLS.

Task 3 — Encryption in Transit (TLS)

Serve a file over HTTPS with a self-signed certificate and confirm the channel is encrypted.

```
# Generate a self-signed certificate
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem \
```

```
    -days 7 -nodes -subj '/CN=localhost'

# Serve HTTPS on port 8443 using a small container
docker run --rm -d --name tls -p 8443:443 \
  -v $(pwd)/cert.pem:/etc/nginx/cert.pem -v $(pwd)/key.pem:/etc/nginx/key.pem \
  -v $(pwd)/record.txt:/usr/share/nginx/html/record.txt nginx

# Connect over TLS (-k accepts the self-signed cert)
curl -k https://localhost:8443/record.txt
```

Security tip: Compare mentally with plain HTTP: over HTTP the record would travel in clear text and any on-path attacker could read it (eavesdropping, Week 3). TLS makes intercepted traffic unreadable.

Note: End of Session A. Stop the TLS container (`docker stop tls`). Keep record.enc, the RSA keys, and all outputs for the report.

Session B (Week 6) — Key Management, Envelope Encryption & Erasure

Start LocalStack if it is not running (see Lab 1). Set `EP='--endpoint-url=http://localhost:4566'`.

Task 4 — Create and Use a KMS Master Key

```
EP='--endpoint-url=http://localhost:4566'

# Create a customer master key (CMK) and capture its KeyId
aws $EP kms create-key --description 'CCSE tenant-A master key'
# Copy the KeyId from the output into KEY_A below
KEY_A=<PASTE_KEYID>

# Encrypt a small secret directly with KMS
aws $EP kms encrypt --key-id $KEY_A --plaintext "$(echo -n 'hello' | base64)" \
    --query CiphertextBlob --output text
```

Task 5 — Envelope Encryption

For large data you do not encrypt with the master key directly. You generate a **data key**, encrypt the data with it locally, and store the data key **wrapped** by the master key. This is envelope encryption.

```
# 5.1 Ask KMS for a data key (returns plaintext + encrypted versions)
aws $EP kms generate-data-key --key-id $KEY_A --key-spec AES_256 \
    --query '[Plaintext,CiphertextBlob]' --output text
# Save column 1 as datakey.b64 (plaintext) and column 2 as datakey.enc (wrapped)

# 5.2 Encrypt the big file locally with the PLAINTEXT data key
base64 -d datakey.b64 > datakey.bin
openssl enc -aes-256-cbc -pbkdf2 -in record.txt -out record.env.enc \
    -pass file:./datakey.bin

# 5.3 Destroy the plaintext data key from disk — keep only the wrapped copy
rm datakey.bin datakey.b64
echo 'Only the KMS-wrapped data key (datakey.enc) remains.'
```

> **Note:** To read the data later you send datakey.enc back to KMS (`kms decrypt`) to unwrap it, use it, then discard it. Only the small master key ever needs hardware-grade protection.

Task 6 — Per-Tenant Keys & Cryptographic Erasure

Create a second tenant key and show that one tenant's key cannot read another's data. Then delete a key to make its data permanently unrecoverable — cryptographic erasure.

```
# A separate key for tenant B
aws $EP kms create-key --description 'CCSE tenant-B master key'
KEY_B=<PASTE_KEYID>

# Schedule deletion of tenant A's key (min window)
```

```
aws $EP kms schedule-key-deletion --key-id $KEY_A --pending-window-in-days 7

# Disable it immediately to simulate erasure
aws $EP kms disable-key --key-id $KEY_A

# Attempt to unwrap tenant A's data key now — it should FAIL
aws $EP kms decrypt --ciphertext-blob fileb://datakey.enc 2>&1 | head -3
```

> **Caution:** Once the key that wrapped the data key is gone, record.env.enc is just noise — no one, not even the provider, can decrypt it. This is why per-object/per-tenant keys make deletion provable (Week 4).

Task 7 — Integrity & Tamper-Evidence

Encryption protects confidentiality; hashing protects integrity. Detect tampering and build a simple hash chain.

```
# Fingerprint the file
sha256sum record.txt

# Tamper with a copy and show the hash changes
cp record.txt tampered.txt; echo 'x' >> tampered.txt
sha256sum record.txt tampered.txt

# Hash chain: each entry includes the previous hash (tamper-evident log)
PREV=0
for line in 'login ok' 'file read' 'export data'; do \
  PREV=$(echo -n "$PREV$line" | sha256sum | cut -d' ' -f1); \
  echo "$line | $PREV"; done
```

Deliverables & Assessment

1. Evidence (label each clearly)

- AES encrypt/decrypt with the MATCH confirmation (Task 1).
- RSA signature **verify** output showing 'Verified OK' (Task 2).
- The `curl -k https://…` output over TLS (Task 3).
- The KMS KeyId(s) and the envelope-encryption steps (Tasks 4–5).
- The failed `kms decrypt` after key erasure (Task 6).
- The two differing SHA-256 hashes and the hash chain (Task 7).

2. Short-Answer Questions

Q1. Compare symmetric and asymmetric encryption: speed, key distribution, and typical use.
Q2. Why is **key management** described as the weakest link, not the algorithm?
Q3. Explain **envelope encryption** and why only the master key needs hardware-grade protection.
Q4. How does **cryptographic erasure** achieve provable deletion where overwriting cannot (in the cloud)?
Q5. How does a **hash chain** make a log tamper-evident (link to tamper-proof logs, Week 6)?

3. Verification Command

```
aws --endpoint-url=http://localhost:4566 kms list-keys
openssl dgst -sha256 -verify public.pem -signature record.sig record.txt
```

Security Best-Practices Checklist

- ☐ Data encrypted at rest (AES) and decryption verified.
- ☐ Asymmetric keys used correctly (encrypt with public, sign with private).
- ☐ Data protected in transit with TLS.
- ☐ Envelope encryption used; plaintext data key not left on disk.
- ☐ Per-tenant keys used; cryptographic erasure demonstrated.
- ☐ Integrity verified with hashing / hash chain.

Cleanup & Teardown

```
docker stop tls 2>/dev/null
rm -f record.* private.pem public.pem key.pem cert.pem datakey.* tampered.txt
docker stop localstack && docker rm localstack
```

Expansion Ideas (Advanced Students)

- Store the master key in a software HSM (**SoftHSM**) and use PKCS#11 to sign — model hardware key protection.
- Stand up **HashiCorp Vault** in a container and use its transit engine for envelope encryption.

- Configure **mutual TLS (mTLS)** between two containers so both sides authenticate.
- Automate key rotation and re-wrap existing data keys under a new master key.

References

- Course lecture — Week 4 (Data Protection); Week 9 (Key Management patterns).
- OpenSSL documentation — www.openssl.org/docs
- AWS KMS concepts (envelope encryption) — docs.aws.amazon.com/kms
- CSA Security Guidance v5 — Data Security & Encryption.